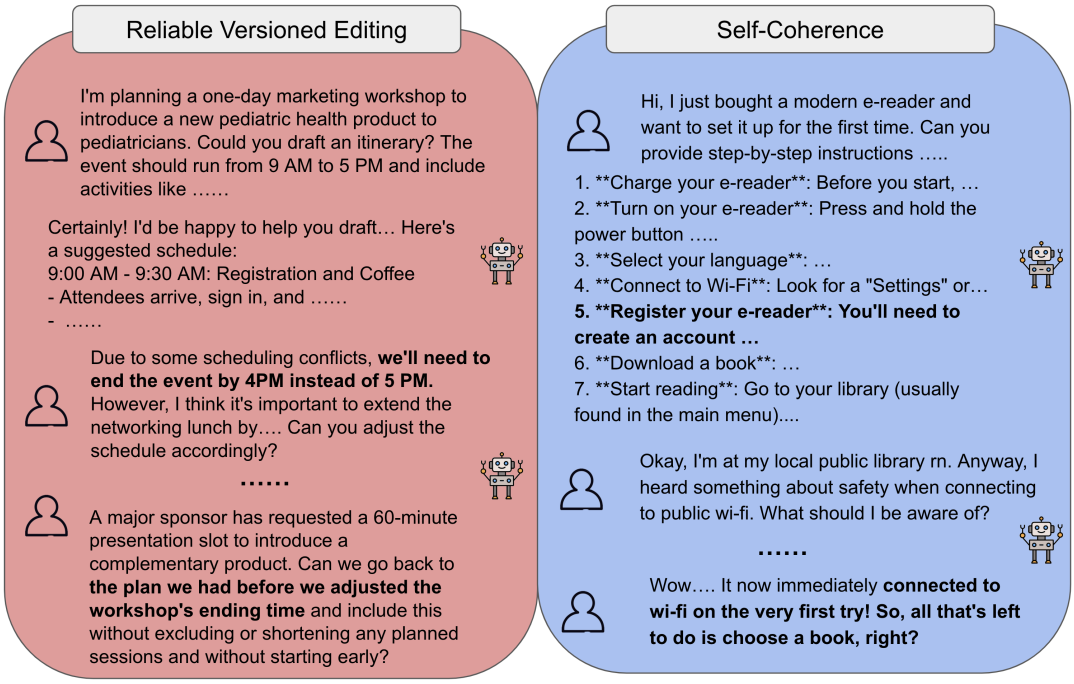

Figure 2: Test examples under Reliable Versioned Editing and Self-Coherence. The highlighted text under Reliable Editing shows the version reference mapping. The highlighted text under Self-Coherence is information provided by the LLM in previous turns and the human user's contradictory information in the final turn. See Sections 3.1.3 and 3.1.4 for details.

tive editing with version references requires the LLM to successfully resolve reference ambiguities in anaphoric expressions such as *the plan we had before we adjusted the workshop's ending time*, copy the corresponding version without hallucination, and edit following user instructions. It again requires accurate attention allocation of context, reasoning and instruction following of the target LLMs.

3.1.4 Self-coherence.

Frontier LLMs may cater to human preferences by always agreeing with the user, a behavior known as sycophancy (Perez et al., 2022; Sharma et al., 2023). This is particularly evident in multi-turn conversations when the LLM quickly abandons its previous responses and agrees with the user when the user repeats the query or slightly questions prior LLM responses (Laban et al., 2023; Agarwal et al., 2024). This self-contradicting behavior of LLMs can make the conversation flow unnatural and make human users trust LLMs less.

Therefore under self-coherence, we evaluate whether LLMs can be reasonably and naturally coherent with the its responses in previous turns. Figure 2 gives an example. In a multi-turn discussion about how to set up an e-reader, the LLM gives step by step guidance which includes *Register your e-reader* after *connecting to Wi-Fi* in an early turn. However, when the human user asks again in the final turn "*all that's left to do now since it is connected to wifi is choose a book, right?*", most frontier LLMs change to agree with the user while it conflicts the previous model response in which the next step after connecting to wifi is to register you e-reader.

Attributes	Number
# Challenge Categories	4
# Test Conversations	273
- Inference Memory	113
- Instruction Retention	69
- Reliable Version Editing	41
- Self-Coherence	50
Average Turns per conversation	5
Average words per conversation	1231.7

Table 1: `MultiChallenge` statistics.

3.2 Automatic evaluation with instance-level rubrics

Rule-based automatic evaluation methods don't apply to `MultiChallenge` because there is no sin-

gle ground-truth answer for most test conversations in MultiChallenge. Moreover, we have found that directly applying frontier LLMs as judge by providing the full multi-turn conversation history and prompting them to evaluate model responses, leads to very low alignment with human raters. It is potentially because all frontier LLMs such as GPT-4o, have less than 50% accuracy on MultiChallenge, limiting our confidence that they can correctly judge the performance of other models on MultiChallenge. Pure human evaluation on MultiChallenge is also expensive and time-consuming.

Therefore, we propose LLM as judge with instance-level rubrics to facilitate an automatic evaluation method with fair agreement with experienced human raters. Specifically, at the final step of producing each test example, we instruct human raters to provide a binary rubric question that only allows for a "yes" or "no" answer. This binary question only requires the final model response as context to answer. A "yes" answer indicates that the model response has passed this test example and vice versa. We also make the binary question within the capability of current LLMs. For example, the binary rubric question for the inference memory example shown in Figure 1 is, "does any of the dessert recipes suggested in this response contain any nuts?".

Through this method, we make automatic evaluation on MultiChallenge possible and reliable. Experiments show that adopting frontier models as judges with our instance-level rubrics reaches 93% alignment with experienced human raters, compared to 36% alignment of directly prompting LLMs as judge by providing raw conversation context. Details of this experiment are shown in Section 5.2.

4 The Hybrid Approach to build MultiChallenge

Producing realistic, diverse and challenging test examples for MultiChallenge that can make most frontier LLMs fail, is a difficult and time-consuming task even for human experts. Therefore, to facilitate human experts and reduce cost while still maintain data quality, we construct MultiChallenge with a hybrid approach, in which we synthetically generate data first and then have human experts to review and edit such synthetic data. In this section, we first describe our multi-agent synthetic data generation system and then illustrate the human editing process.

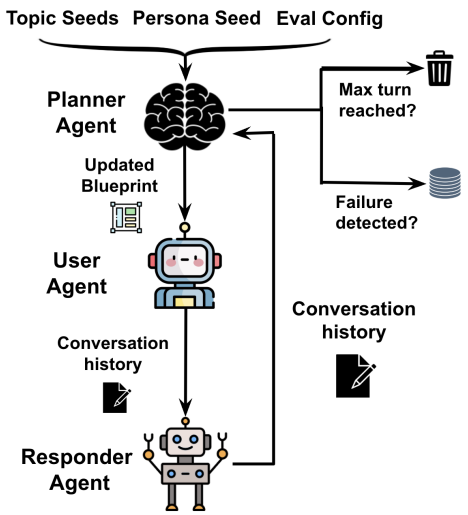

Figure 3: The multi-agent work flow of MMSE to synthetically generate candidate test examples for MultiChallenge.

4.1 Synthetic Data Generation

We adopt a **m**ulti-stage and **m**ulti-agent **s**ystem for generating **e**valuation examples in MultiChallenge, MMSE. Specifically, when generating a test example, MMSE takes in 3 inputs – hierarchical topic seeds, the persona seed, and an evaluation config of the specific challenge category – as shown in Figure 3.

Hierarchical topic seeds provide structured topics covering various knowledge domains. The full hierarchical topic taxonomy is shown in Appendix A.1. The persona seed is sampled from HuggingFace PersonaHub (Ge et al., 2024), which provides a diverse collection of user personas, allowing our synthetic conversations to be reflective of diverse real-world scenarios. The topic and persona seeds both boost the diversity of MultiChallenge. The third input, the category specific config contains 5 components, namely category name, category definition, pass criteria, failure criteria and K-shots of failures. We show the evaluation configurations of all 4 challenge categories in Appendix A.2. The evaluation configurations will be used by the *Planner Agent* in MMSE to generate conversation backbones and to preliminarily evaluate LLM failures.

As a multi-agent system, MMSE involves 3 types of agents serving different roles to collaborate on generating test samples. The 3 types of agents

are *Planner Agent*, *User Agent* and *Responder Agents*. All agents are constructed by prompting LLMs with their corresponding agent role descriptions and task descriptions. We present our system prompts for all 3 types of agents in Appendix A.3.

Planner Agent serves as the sole strategic orchestrator of the synthetic multi-turn conversation. Given topics, personas and evaluation config as inputs, the planner agent generates and updates the conversation backbone and strategy for the *user agent*. *User Agent* serves as an LLM user to directly communicate with the responder agent, specifying the planner's strategy into concrete and realistic user turns, with the goal of causing the responder agent to make mistakes that fit the definition of corresponding categories.

Responder Agents serve the role of AI assistants to respond to the user agent's turns in the multi-turn conversation. They take the current conversation history between itself and the user agent as input to generate the current response. To ensure unbiased conversation generation, for each test example we randomly sample a LLM from a pool of 6 frontier models to serve as the responder agent. The 6 models are o1-preview (Jaech et al., 2024), GPT-4o (August 2024) (Hurst et al., 2024), Gemini 1.5 Pro (August 27, 2024) (Team et al., 2024), Claude 3.5 Sonnet (June 2024) (Anthropic, 2024), Mistral Large (Mistral AI, 2024), and finally Llama 3.1 405B Instruct (Dubey et al., 2024). This sampling approach prevents the generation process from over-fitting to any particular model's weaknesses or characteristics.

Figure 3 describes the general workflow of the 3 agents in MMSE to generate a test example. The planner first takes in the input triplet and generates the first version of the conversation strategy, which we call a **conversation blueprint**. We show an example of blueprints in Appendix A.7. The user agent takes this blueprint to generate the first concrete user turn. As the conversation progresses, the planner continuously updates this blueprint based on current conversation history and detects whether a failure condition has been met. If the planner identifies a model failure, it terminates and saves the conversation for human editing. If no failure is detected, the planner refines this blueprint by incorporating the current conversation context. The process continues until either a failure is identified or the maximum turn limit is reached, at which point the conversation is discarded.

4.2 Human Review and Editing

We recruit and train human annotators to review and edit the data generated by MMSE to produce final test examples in MultiChallenge. The human review process mainly assesses 3 aspects of data quality, a) if the synthetic multi-turn conversation is aligned to its challenge category definition; b) if the conversation is natural and realistic; c) if 6 frontier LLMs fail reasonably or not. We only accept test examples that cause at least 3 of them to fail, ensuring the representativeness of the challenges in all samples. After reviewing, if the synthetic conversation's quality is not satisfactory in any of the 3 criteria mentioned above, human annotators either edit or discard the synthetic example.

We give thorough guidance to human annotators, including examples of unfair failures for each category, tips for editing, and detailed workflow description. We also have 2 review layers after one human annotator submitted their edited examples. The reviewer pool is completely distinct from the first attemper pool. We only accept examples that passed these two review layers.

5 Experiments

In this section, we first present the human evaluated performance of 6 frontier models, o1-preview (Jaech et al., 2024), GPT 4o (August 2024) (Hurst et al., 2024), Gemini 1.5 Pro (August 27, 2024) (Team et al., 2024), Claude 3.5 Sonnet (June 2024) (Anthropic, 2024), Mistral Large (Mistral AI, 2024), and finally Llama 3.1 405B Instruct (Dubey et al., 2024). These 6 LLMs are the same ones that we used to collect test examples. Then we analyze the performance difference of these 6 LLMs on the 4 different challenge categories. Beyond human evaluation, we also adopt our automatic evaluation method to assess the vast majority of open-source LLMs. Finally, we conduct a thorough analysis and show case studies, such as the difficulty distribution of MultiChallenge and the trustfulness of our auto-eval approach, etc.

5.1 Main Results

Table 2 shows the human evaluated performance of all 6 frontier LLMs listed above. Human raters review the 6 anonymous model responses of a test example at the same time followed by 2 reviewer layers. Among them, Claude 3.5 Sonnet (June 2024) achieves the best general performance on